

Student Enrollments BI Solutions Admin Manual

Certificate in Business Intelligence and Data Integration

Change Log: 2025/12/11, Solsen, Created first draft of document

Introduction

The BI solution is designed for a private school of less than 30 employees. The purpose is to track and report on student enrollments at the school.

Purpose of This Document

This manual should be used to understand the entire business intelligence solution – from source data to reporting. Each element of the solution will be outlined. This should be referenced to understand the current solution, and when creating additional iterations of the solutions.

Scope

The design of the solution is visualized below. The end goal is to use student enrollment data in order to develop professional & user-friendly reports. The report will be used by employees of the school (teachers, administrators, etc.).

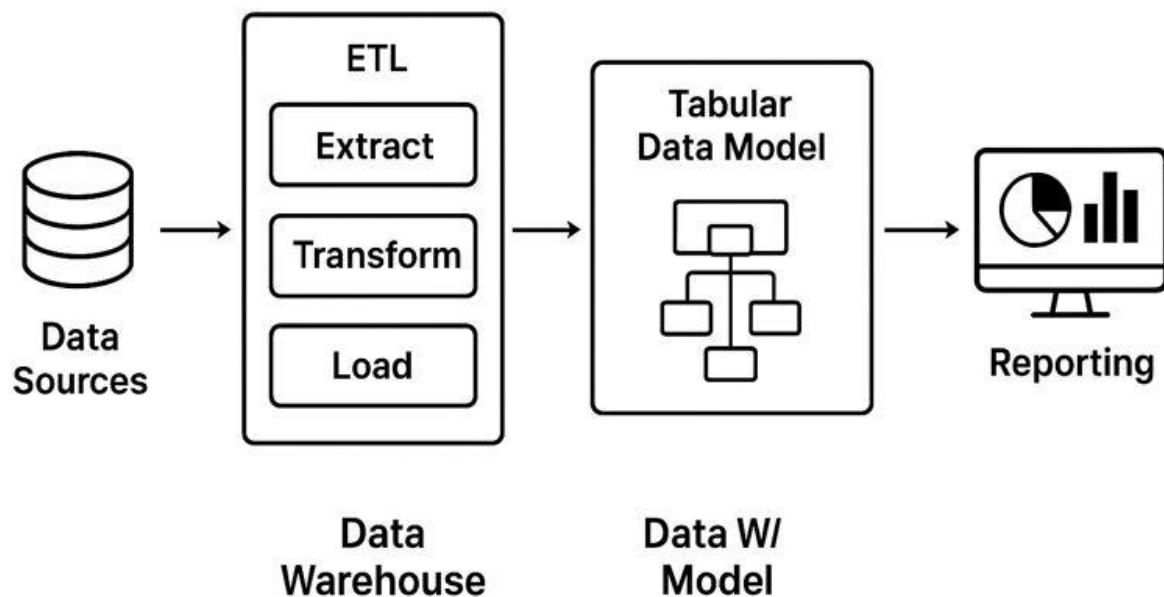


Figure: Environment Overview

- **Organization Type:** Small private school
 - **Employee Count:** <30
 - **Primary Stakeholders:** Administration, Finance, Academic Staff
 - **Technologies Used:**
 - SQL Server / Azure SQL
 - Microsoft Report Builder
 - SSAS Tabular Model
 - Excel for Reporting
-

The Visual Studio Solution

Solution Overview

The BI solution is developed and maintained using a single Visual Studio (SSDT) solution. This ensures source control, consistency, and ease of deployment.

Solution Components

The solution contains the following projects:

- SQL Server Database Project – Data Warehouse schema and views
- Integration/ETL Scripts – Stored procedures and ETL logic
- SSAS Tabular Project – Semantic model and DAX measures

Deployment Targets

- SQL Server instance hosting DWStudentEnrollments
 - SSAS instance hosting the Tabular Model
-

The Source Database

Source System Overview

The source system is an Azure SQL database named StudentEnrollments. It is normalized and optimized for transaction processing.

Source Entities

Key source tables include: Students, Classes, Enrollments, Classrooms, Departments

Entity Relationship Diagram (ERD)

The following ERD represents the snowflake schema for the StudentEnrollments OLTP Database:

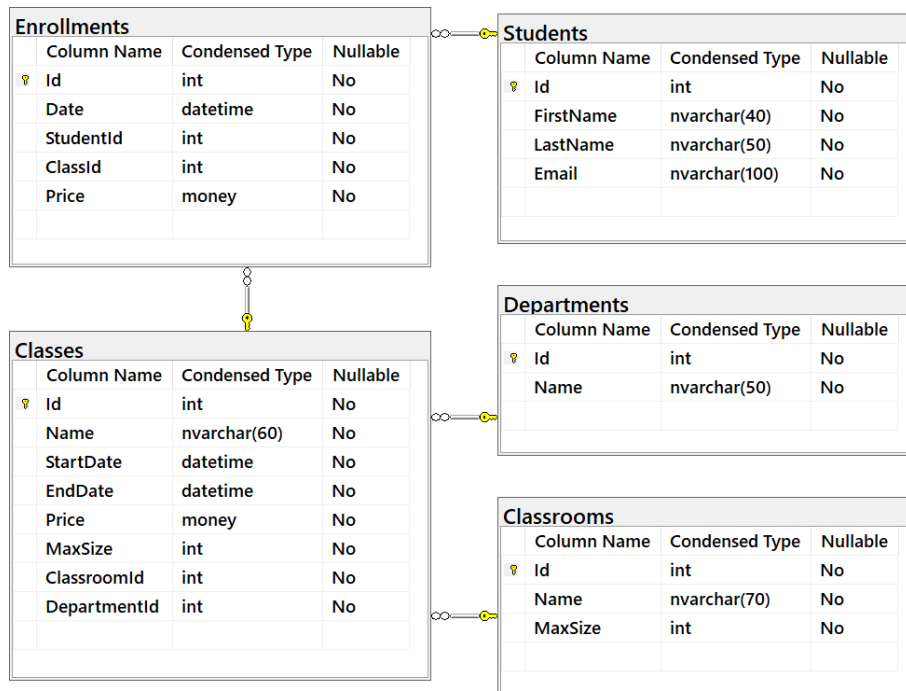


Figure: ERD for the source database

Source Design Considerations

- Highly normalized (snowflake-style)
- Uses natural keys
- Optimized for OLTP, not reporting
- Requires transformation before analytics

The following figure shows an example of the code to create a table in the source database.

```
-- Fact Table --
Create Table Enrollments
( Id int Not Null Identity Constraint pkEnrollments Primary Key
, [Date] datetime Not Null
, StudentId int Not Null
, ClassId int Not Null
, Price money Not Null
);
Go
```

Figure: Source Table creation code

Data Warehouse Design and Creation

The Data Warehouse is designed to include 4 tables. These tables are aligned in a star schema pattern. There is 1 fact table surrounded by 3 single-table dimensions. The star design is used to enhance simplicity over the snowflake design found in the source database. The 'Departments' and 'Classrooms' tables have been collapsed with the 'Classes' table to form one combined table in the data warehouse.

Data Warehouse Overview

The Data Warehouse uses a streamlined design allowing for more efficient querying and reporting. The data from this database will be directly used to build reports. For this reason, entity names have been simplified. Entity names are more intuitive in the data warehouse, so users can easily identify their purpose.

Data Warehouse Metadata Worksheet

Purpose

The metadata worksheet documents table structures, columns, data types, and nullability.

Usage

- Developer reference
- Data governance
- Impact analysis

Entity Relationship Diagram (ERD)

The following ERD represents the final star schema for the DWStudentEnrollments Data Warehouse. It includes: DimClasses, DimStudents, DimDates, FactEnrollments

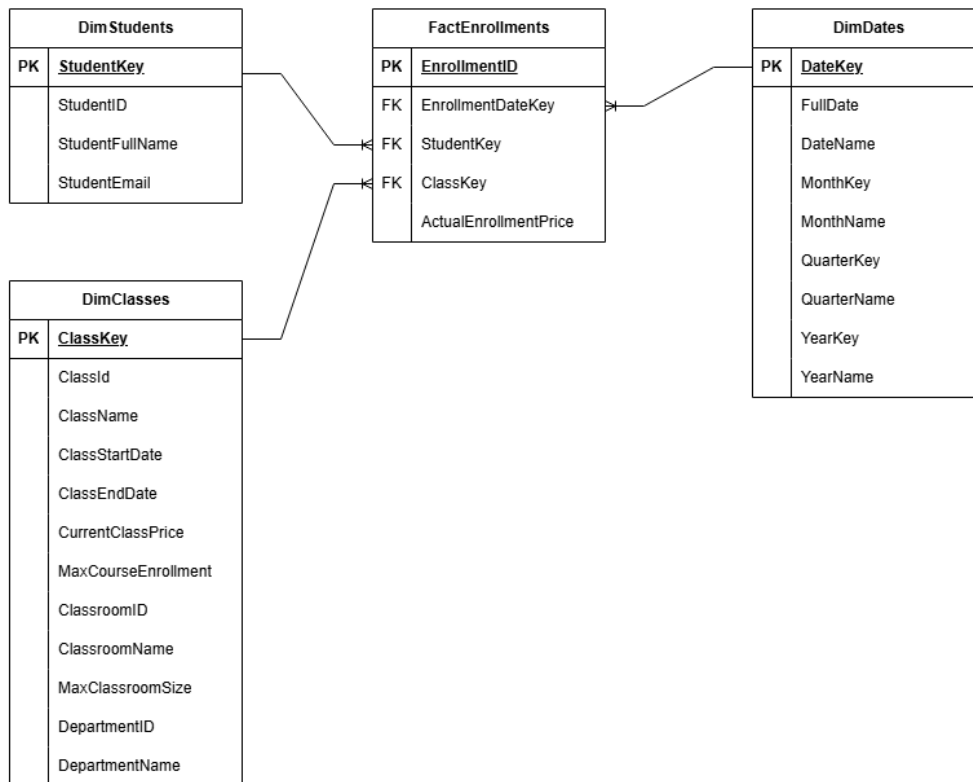


Figure: ERD for the Data Warehouse

Star Schema Overview

The data warehouse follows a **star schema** consisting of one fact table (FactEnrollments) and three supporting dimensions (DimClasses, DimStudents, DimDates).

Dimension Tables

DimClasses

- **Grain:** One record per unique class
- **Key:** ClassKey (surrogate)
- **Attributes:** ClassID, ClassName, ClassStartDate, ClassEndDate, CurrentClassPrice, MaxCourseEnrollment, ClassroomID, ClassroomName, MaxClassroomSize, DepartmentID, DepartmentName

DimStudents

- **Grain:** One record per unique student
- **Key:** StudentKey (surrogate)
- **Attributes:** StudentID, StudentFullName, StudentEmail

DimDates

- **Grain:** One record per calendar date
- **Key:** DateKey
- **Attributes:** FullDate, DateName, MonthKey, MonthName, QuarterKey, QuarterName, YearKey, YearName

Fact Tables

FactEnrollments

- **Grain:** One record per student enrollment in a class on a given date
- **Natural Keys:** EnrollmentID, EnrollmentDateKey, StudentKey, ClassKey
- **Measures:** ActualEnrollmentPrice
- **Foreign Keys:** Links to DimClasses(ClassKey), DimStudents(StudentKey), DimDates(DateKey)

Abstraction Layer (Views)

A set of base views has been created to abstract underlying table structures:

- vDimClasses
- vDimStudents
- vDimDates
- vFactEnrollments

A metadata view (vMetaDataStudentEnrollments) provides system-level documentation of tables, columns, data types, and nullability.

Physical Implementation: Example DW Table Creation Script

The following image illustrates an example of the SQL code used to create a Data Warehouse dimension table (DimClasses). This visual reference supports understanding of structure, constraints, and field definitions.

```
-- *****
-- Create the Tables
-- *****

Create Table DimClasses
( ClassKey int Identity          Not Null
, ClassID int                   Not Null
, ClassName nvarchar(100)       Not Null
, ClassStartDate date           Not Null
, ClassEndDate date            Not Null
, CurrentClassPrice money       Not Null
, MaxCourseEnrollment int      Not Null
, ClassroomID int              Not Null
, ClassroomName nvarchar(100)  Not Null
, MaxClassroomSize int         Not Null
, DepartmentID int             Not Null
, DepartmentName nvarchar(100) Not Null
Constraint pkDimClasses Primary Key (ClassKey)
);
Go
```

Figure: Data Warehouse table creation code

ETL Processing

ETL Architecture Overview

The ETL solution is implemented fully in SQL Server using:

- A linked server connection to the Azure operational database (AzureLinkedServerName)
- ETL staging views for each dimension and fact
- Metadata logging (EtLog table, vEtLog view, plnsEtLog procedure)
- Stored procedures that manage load sequencing, error handling, and re-creation of foreign keys

The ETL flow follows this sequence:

1. Create metadata tables/views (EtLog)
2. Drop foreign keys to enable bulk reload
3. Truncate all DW tables
4. Load DimDates
5. Load DimStudents
6. Load DimClasses

7. Load FactEnrollments
8. Re-create foreign keys

Linked Server Configuration

The ETL process connects to the Azure SQL database using a linked server:

- Linked server name: **AzureLinkedServerName**
- Provider: **MSOLEDBSQL** (modern OLE DB driver)
- Target database: **StudentEnrollments**
- Authentication: SQL login configured via sp_addlinkedsrvlogin

The linked server enables direct SELECT queries against Azure tables such as:

- Students
- Classes
- Enrollments
- Classrooms
- Departments

ETL Metadata & Logging

A full auditing system is implemented:

EtlLog table – Logs every ETL event, including errors. **vEtlLog view** – Provides formatted date/time for readability. **plnsEtlLog procedure** – Inserts log entries with TRY/CATCH error handling.

This provides:

- Full traceability of every ETL step
- Row counts for each load event
- Captured SQL Server error messages

Pre-Load Tasks

Two stored procedures prepare the DW for a fresh reload:

pETLDropFks

- Drops FK constraints on FactEnrollments
- Ensures load operations do not fail due to referential restrictions

pETLTruncateTables

- Truncates: FactEnrollments, DimClasses, DimStudents, DimDates
- Ensures a clean, full refresh each ETL cycle

Dimension Loads

ETL View and Load Examples

The following images illustrate examples of the ETL process used to populate dimension tables.

ETL View Creation Example (vETLDimStudents)

This image shows how source data is transformed and exposed through an ETL staging view.

```
/******dbo.DimStudents*****/  
  
Create or Alter View vETLDimStudents  
As  
    Select  
        [StudentId]    = s.Id  
        ,[StudentFullName] = Cast((s.FirstName + ' ' + s.LastName) as nVarChar(200))  
        ,[StudentEmail]= s.Email  
    From AzureLinkedServerName.StudentEnrollments.dbo.Students As s;  
Go
```

Figure: ETL view creation code

ETL Insert Procedure Example (pETLDimStudents)

This image demonstrates the stored procedure that loads transformed data into the DimStudents table, including logging and error handling.

```
Create Or Alter Proc pETLDimStudents  
As  
Begin  
    Declare @RC int = 0;  
    Declare @Message varchar(1000)  
    Begin Try  
        Begin Tran;  
        Insert Into DimStudents  
        (StudentID,StudentFullName, StudentEmail)  
        Select  
            [StudentID]  
            ,[StudentFullName]  
            ,[StudentEmail]  
        From vETLDimStudents  
        Set @Message = 'Filled DimStudents (' + Cast(@@RowCount as varchar(100)) + ' rows)';  
        Commit Tran;  
        Exec pInsETLLog  
            @ETLAction = 'pETLDimStudents'  
            ,@ETLLogMessage = @Message;  
        Set @RC = 1;  
    End Try  
    Begin Catch  
        If @@TRANCOUNT > 0 Rollback;  
        Declare @ErrorMessage nvarchar(1000) = Error_Message();  
        Exec pInsETLLog  
            @ETLAction = 'pETLDimStudents'  
            ,@ETLLogMessage = @ErrorMessage;  
        Set @RC = -1;  
    End Catch  
    Return @RC;  
End  
Go
```

Figure: ETL data load into data warehouse code

Dimension Loads

Dimension loads use ETL views that pull and transform data from Azure.

DimDates (pEtldimDates)

DimDates is generated internally using a WHILE loop (2019–2029). Includes:

- Complete calendar build
- Additional records: Unknown (-1), Corrupt (-2)
- Calculated columns: DateName, MonthName, QuarterName, YearName

DimStudents (vETLDimStudents / pETLDimStudents)

Key transformations:

- Concatenates FirstName + LastName → StudentFullName
- Passes StudentID and Email directly

DimClasses (vETLDimClasses / pETLDimClasses)

Key transformations:

- FULL JOIN across Classes, Classrooms, and Departments
- ISNULL rules assign defaults such as:
 - Undecided (for missing names)
 - -1 surrogate substitute for missing IDs
 - Default dates (1900-01-01 / 1900-01-02)
- Captures class metadata (price, max size, classroom capacity)

Fact Table Load

The fact ETL uses lookups to the populated dimensions:

vETLFactEnrollments performs:

- Joins Enrollments → DimDates via enrollment date
- Joins Enrollments → DimStudents via StudentID
- Joins Enrollments → DimClasses via ClassID
- Produces EnrollmentDateKey, StudentKey, ClassKey for fact load

pETLFactEnrollments inserts:

- EnrollmentID
- EnrollmentDateKey
- StudentKey
- ClassKey
- ActualEnrollmentPrice

Post-Load Tasks

After all tables are populated, foreign keys are re-established using:

pETLReplaceFks

- FK: FactEnrollments → DimClasses
- FK: FactEnrollments → DimStudents
- FK: FactEnrollments → DimDates

ETL Pipeline Inventory

Object	Type	Purpose
EtlLog	Table	Stores ETL metadata and error logs
vEtlLog	View	Human-readable ETL log formatting
pInsEtlLog	Sproc	Centralized logging routine
pETLDropFks	Sproc	Drops fact table FK constraints
pETLTruncateTables	Sproc	Clears all DW tables
pEtlDimDates	Sproc	Generates full DimDates table
vETLDimStudents	View	Transforms student data from Azure
pETLDimStudents	Sproc	Loads DimStudents
vETLDimClasses	View	Combines class, classroom, and department data
pETLDimClasses	Sproc	Loads DimClasses
vETLFactEnrollments	View	Performs surrogate key lookups
pETLFactEnrollments	Sproc	Loads FactEnrollments
pETLReplaceFks	Sproc	Recreates FK constraints

Tabular Model

Data Sources

The Tabular Model retrieves data directly from Data Warehouse tables and ETL Tabular Views. Data Warehouse tables are loaded into views as outlined below. The views offer as an abstraction layer, and a method to perform final data transformations before loading data to the tabular model.

- DimDates → vTabularETLDimDates
- DimStudents → vTabularETLDimStudents

- DimClasses → vTabularETLDimClasses
- FactEnrollments → vTabularETLFactEnrollments

Tabular ETL View Example

The following image shows an example of a view used specifically for loading data into the Tabular Model. This view converts the EnrollmentDateKey into a proper date format for analysis:

```
-- Create the FactEnrollments view
Create or Alter View vTabularETLFactEnrollments
AS
Select
    [EnrollmentID]
    ,[EnrollmentDateKey] = Cast(Cast(EnrollmentDateKey as varchar(10)) as date)
    ,[StudentKey]
    ,[ClassKey]
    ,[ActualEnrollmentPrice]
From FactEnrollments
Go
```

Figure: Tabular ETL view creation code

Model Structure

Tabular Model Diagram

The following diagram represents the full tabular model implemented for the BI solution.

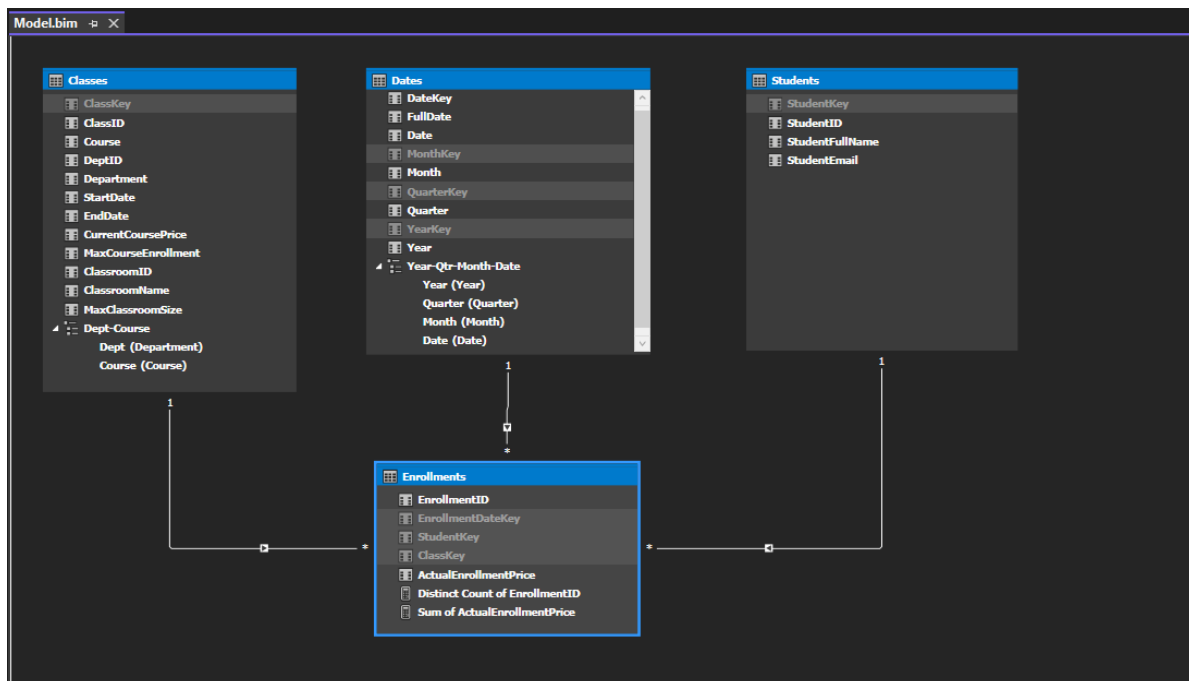


Figure: Tabular model ERD diagram

The full tabular model includes:

- Classes dimension
 - Dates dimension
 - Students dimension
 - Enrollments fact table
 - Relationships between all tables
 - Calculated Columns
 - Measures (DAX)
-

5. Sample Reports

5.1 Reporting Overview

The Sample Reports are used to test the final result of our solution. We connect to excel directly from the Tabular model to ensure the measures translate accurately. We also connect to excel from the Data Warehouse directly to ensure that connection is possible.

Two Reports Generated based on the following connections:

- Tabular to Excel
- SQL Server Data Warehouse to Excel

If the connections run smoothly then this ensures reporting will be efficient after more data is added to the model.
